Projects: High-Performance Computing, SS26

Create an OpenCL program that reads an image from a file, applies a Gaussian blur to it and stores the result back to an output image file. A Gaussian blur is a very common image filter. There should be plenty of information and implementation details online (e.g. https://en.wikipedia.org/wiki/Gaussian_blur). Please do not hesitate to approach me if you have any problems finding appropriate sources online.

Simple Gaussian Blur (40 points)

- Implement a Gaussian blur as a **single kernel call**
- Deadline is the **22nd of May 2026 23:59**

Optimized Gaussian Blur (30 points)

- Implement the Gaussian blur as a **two-pass filter**
- Implement a **single kernel function** and call it **two times**.
- One call should run **column-wise** and one call should run **row-wise (Do not transpose the image!)**
- **The work group size should always be equal to one column / one row** of the source image (adjust the image size if your device capabilities do not allow for a certain size)
- **Exit the program if the resulting work-group is too big** for the system
- Minimize access of **source pixel values** from Global Memory by utilizing **Local Memory**
- Deadline is the **12th of June 2026 23:59**

General Requirements

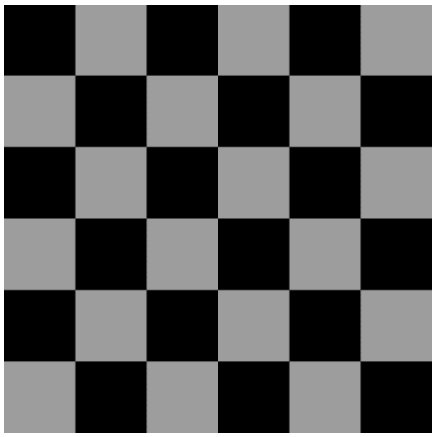
- **Each kernel invocation** should only write a **single pixel value** to the output image
- The Gaussian filter kernel can have a **variable size** (up to size 9) and should be provided as a **kernel argument**
- The resulting image should **preserve** its overall **brightness** after applying the filter
- **Do not use the image memory object!** Utilize normal buffer memory objects
- Beware of borders, and **clamp sampling to the nearest valid pixel** (see example in side notes)
- Use a **2-dimensional NDRange** (using width and height of source image)
- The blur should also work with images where **width != height**
- **Query device capabilities** to make sure the NDRange is valid for the system
- Make sure to test your program thoroughly, it **has to run on my machine without any changes**!
- Code cleanliness will influence grading, so make sure to **tidy up your code**!
- Upload a .zip file containing the complete Project with all required files/libraries to Moodle
- Implement the Program in one of the following
    - C/C++ using the official OpenCL C API. **The OpenCL C++ Bindings are not allowed!**
    - C# using OpenCL.NET https://www.nuget.org/packages/OpenCL.Net/

- Java using JOCL http://www.jocl.org/downloads/JOCL-0.2.0-RC-bin.zip
- **Generally: It is not allowed to use any APIs or libraries that significantly reduce OpenCL coding complexity**!

Side Notes

- You can freely choose the source image format
- Java (BufferedImage) and C# (Bitmap) provide already classes to read/write images. For C++ you can find a .tga image reader/writer in Moodle (but you can use whatever library you want)
- Memory coalescing and local memory bank conflicts (see Unit 5) do not need to be considered
- The kernel filter values do not need to be calculated and can be hardcoded in your program.
  You can generate your own Gaussian filter kernels (2D filter for simple and separable filter for optimized version) using the generator program found in Moodle. Kernel size and blur strength is adjusted by changing the appropriate defines. To run the program, you can simply copy-paste it into an online Cpp Debugger (e.g.: https://www.onlinegdb.com).
- Example for border handling: The image below has a width and height of 6 pixels. Valid pixel coordinates are thus between [0, 0] and [5,5] (red box). If you need to sample a pixel outside this area, you should instead use the nearest valid pixel. E.g. All samples in the green box (which are outside the valid image area) should use the pixel value of [0,2] instead.

Original image



Values used for samples outside valid pixels